

Solution Proposal

Topic	Bot Prevention
Prepared by	Ronald Thian
Prepared on	23 September 2025
Approved by	
Approved on	

Context

Leading up to 9th September 2025, it was discovered that legitimate users of the National Lottery backoffice system had been running their own scripts/bots to query mass amounts of data, adding unexpected stress onto our infrastructure.

Problem Evaluation

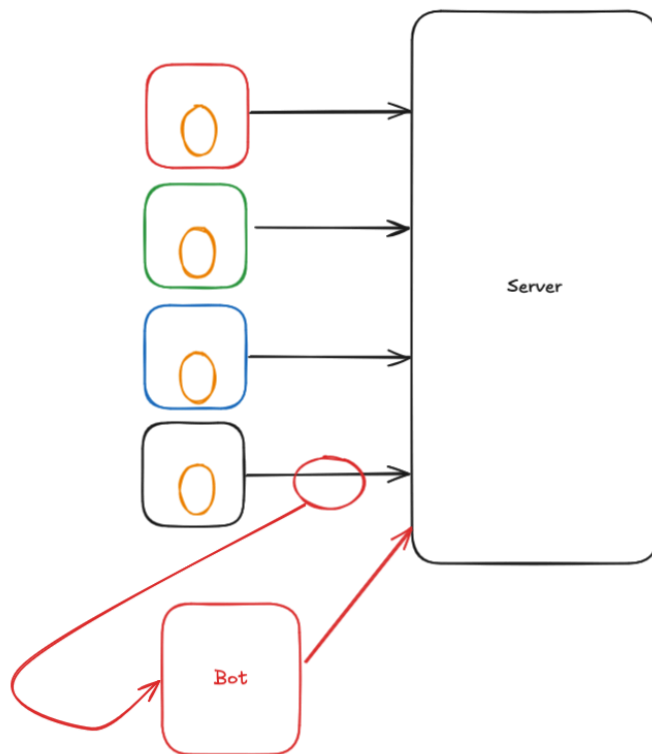
Several report pages in the backoffice system were custom designed to fit our clients' needs and could query large amounts of data. However, in order to skip development time needed to raise and develop a feature, users were found running their own scripts. This resulted in an inefficient flood of queries via multiple simultaneous calls of existing endpoints, which could have been prevented with an optimized endpoint.

Temporary Preventive Measures and Hotfix

The identified users were reminded several times at different occasions about the repercussions of their actions and guided towards the proper feature request flow, but to no avail.

Current Setup - Client-side Fingerprint IDs

A client side library, fingerprintjs [1], is used currently, to be passed within the header into the server. However, the server only checks for the existence of this key and does no comparison or validation, meaning, anyone could easily sniff our website, copy a fingerprint id, and paste into their scripts.



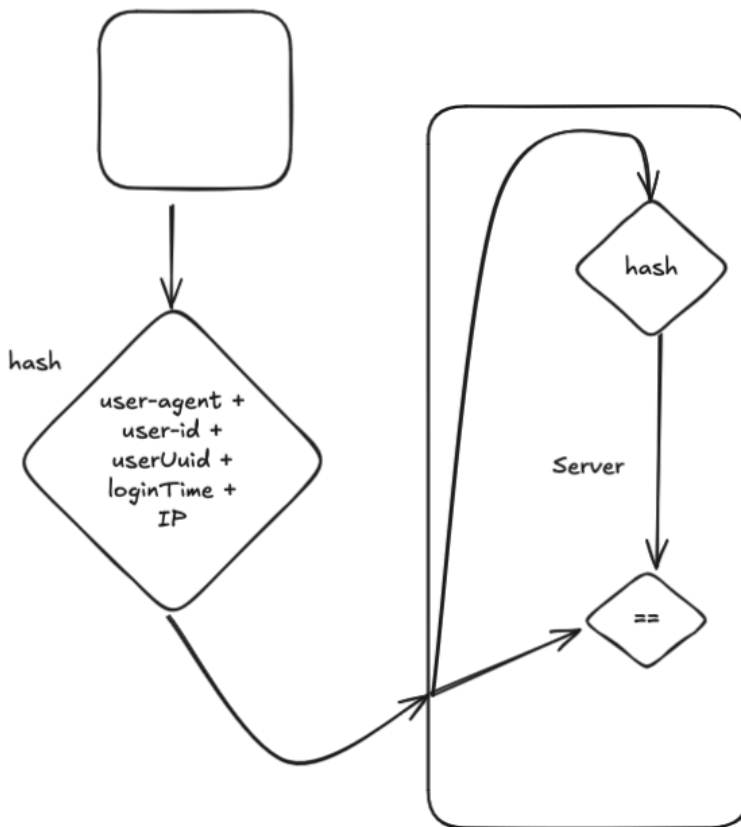
Although this setup allowed us to have a basic level of filtering, it is still extremely easy to work around.

Proposed Solution

Solution 1 - Hash Comparison

Similar to digital certificates, a quick hashing algorithm can be implemented to validate requests. For example, a custom formed string consisting of unique request details, can be hashed and passed to the server.

1. Frontend hashes a uniquely formed string, containing values like `userId`, `user-agent`, etc.
 - a. E.g. `md5(`${userId}+${user-agent}+${loginTime}+...`)`.
2. Frontend passes this hash as header
3. Server calculates its own hash using the request details identical to how Frontend calculates it
4. Server compares calculated hash with request header hash



This would allow the server to block requests that are using hashes copied from a browser, into somewhere not originally intended.

Solution 2 - Fingerprint Pro

Fingerprint provides a SaaS, Smart Signals [2], that allows our system to confidently block bots, however, this requires the procurement of an external service.

1. Frontend calls Fingerprint SaaS endpoint and passes key visitor information
2. Frontend calls Server endpoint with `visitorId`
3. Server calls Fingerprint SaaS with `visitorId` and verify request details with incoming request
4. Server validates request

Recommendation

Recommended Solution - Solution 1, Hash Comparison

Fingerprint Pro has a suite of features and adds a subscription cost to our current solution, as we do not require most of the features, implementing a simple hash comparison feature akin to Digital Certificate allows us to have more control over the application while adding difficulty for botting.

It must be noted that this does not eradicate bots completely, but adds a layer of complexity over our endpoints, preventing most layman bot/scripts.

Although it also adds compute requirements to endpoints, a quick hashing algorithm like md5 should come at a negligible cost.

References

1. FingerprintJS, Inc. (n.d.). *Identify every visitor & mobile device*. Retrieved 23 September 2025, from <https://fingerprint.com/>
2. FingerprintJS, Inc. (n.d.). *Smart Signals*. Retrieved 23 September 2025, from <https://fingerprint.com/products/smart-signals/>